# 1. 命名规范

## 1.1 文件与目录命名

- **Python 文件（模块）与目录**：使用全小写字母，单词之间使用下划线分隔（Snake Case）。例如：`app.py`，`model_service.py`，`streamlit_app.py`。
- **资源与配置文件**：保持小写和下划线，例如：`best_fold_model.pt`，`requirements.txt`。

## 1.2 代码标识符命名

- **类名（Classes）**：使用首字母大写的驼峰命名法（Pascal Case）。例如：`class EssayScorer:`，`class HistoryManager:`。
- **函数、变量与方法（Functions, Variables, Methods）**：使用全小写字母加下划线（Snake Case）。例如：`def predict_score(essay_text):`，`submission_id = 1`。
- **常量（Constants）**：使用全大写字母加下划线。例如：`DEFAULT_MODEL_PATH = "best_fold_model.pt"`，`MAX_SEQ_LENGTH = 512`。

---

# 2. 代码格式与排版

## 2.1 缩进与行宽

- **缩进**：统一使用 **4 个空格**，严禁使用 Tab 键或混用 Tab 与空格。
- **最大行宽**：限制在 **79 - 120 字符** 之间。对于超长的模型参数调用或长字符串，推荐使用括号进行隐式换行。

## 2.2 导入规范 (Imports)

所有的 `import` 语句应放在文件顶部，并按照以下顺序分组，组与组之间空一行：

1. Python 标准库（如 `os`，`json`，`sqlite3`）。
2. 第三方库（如 `torch`，`flask`，`streamlit`，`transformers`）。
3. 本地项目模块（如 `from backend.model_service import ModelService`）。

Python

```
import os
import sqlite3

import torch
import pandas as pd
from flask import Flask, request
from flask_restful import Api, Resource

from backend.storage import save_to_history
```

---

# 3. 架构设计与模块规范

在系统核心逻辑的编写中，强烈推荐采用**面向对象设计原则 (Object-Oriented Design)** 封装数据处理流、API 资源和模型推理逻辑。避免在单个文件中编写过程式的冗长脚本。

## 3.1 后端 API 规范 (Flask-RESTful)

- **资源封装**：将每一个 RESTful API 端点封装为独立的 `Resource` 类，清晰拆分 `get`, `post`, `delete` 等方法。
- **状态码响应**：严格遵循 HTTP 状态码语义返回结果（如 200 OK，201 Created，400 Bad Request，500 Internal Server Error）。
- **全局异常处理**：在 Flask 应用顶层捕获模型推理崩溃或数据库死锁异常，避免向前端暴露敏感的报错堆栈。

## 3.2 深度学习模型规范 (PyTorch & RoBERTa)

- **设备无关性**：编写模型推理代码时，必须动态识别设备，避免硬编码 CPU 或 GPU。

  Python

  ```
  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
  model.to(device)
  ```

- **内存管理**：在进行模型推理时（无需反向传播），必须使用 `with torch.no_grad():` 上下文管理器，以减少显存和内存占用，防止线上服务 OOM (Out of Memory)。

## 3.3 前端规范 (Streamlit)

- **UI 与逻辑分离**：尽量将数据处理逻辑（如 CSV 解析、指标计算）提取到单独的辅助函数中，Streamlit 主脚本应只负责渲染组件。
- **状态管理**：合理使用 `st.session_state` 保存分页进度、用户选择的题目类型和历史记录查询条件，避免页面刷新导致状态重置。

---

# 4. 注释与文档

## 4.1 函数与类文档字符串 (Docstrings)

所有公有模块、类和方法必须提供完整的 Docstring，说明功能、参数、返回值及可能抛出的异常。推荐使用 Google 风格：

Python

```
def extract_features(text: str, max_length: int = 512) -> torch.Tensor:
    """
    使用 RoBERTa 提取文本特征向量。

    Args:
        text (str): 输入的作文文本。
        max_length (int, optional): 截断的最大长度。默认值为 512。

    Returns:
        torch.Tensor: 提取出的文本特征向量，形状为 [1, hidden_size]。
    """
    pass
```

### 4.2 行内注释

- 对复杂逻辑（如正则匹配规则、张量维度变换 `view`/`reshape`、数据库复杂的连表查询）必须加行内注释，解释“为什么这么做”而不是“代码在做什么”。

---

## 5. 存储与数据处理规范

- **数据库连接**：使用 SQLite (如 `history.db`) 时，确保数据库连接的正确开启和关闭，推荐使用 `with` 上下文管理器或确保在请求结束时释放资源。
- **文件上传安全**：处理批量评分上传的 `.csv` 文件时，必须使用 `werkzeug.utils.secure_filename` 对文件名进行安全过滤，防止目录遍历攻击。归档至 `uploads/` 目录时，应使用时间戳重命名以避免文件覆盖。

---

## 6. 依赖管理与版本控制

- **依赖锁定**：新增第三方库时，必须及时更新 `requirements.txt`，并明确指定大版本号甚至小版本号（例如 `torch>=2.2.0`, `transformers>=4.40.0`），保证部署环境的一致性。
- **Git 提交信息**：提交日志应简明扼要，格式要求为 `[标签] 描述`：
    - `[Feat]`：新增功能（如：支持批量 CSV 评分导出）
    - `[Fix]`：修复 Bug（如：修复长文本导致 RoBERTa OOM 的问题）
    - `[Docs]`：文档更新
    - `[Refactor]`：代码重构（如：使用面向对象方式重写数据库持久化模块）